

PokéTrader

Team 3

Team Member 1:

Joseph Ko (Team Lead)

joseph.ko@sjsu.edu

Team Member 2:

Isaiah Mak

isaiah.mak@sjsu.edu

Team Member 3:

Thaneesha Singh

thaneesha.singh@sjsu.edu

Functional Requirements

PokéTrader offers many different services for users. Every 12 hours, users can redeem a ticket to open a free Pokémon pack of any available set. Users can also check their virtual binder to see cards that they've already collected. To search for specific cards, users can type into the search bar to look for cards with specific characteristics (name, types, rarities). Users can also push out requests for trades and those requests can be accepted by other users who are interested in trading for particular cards. During a trade, users can chat with each other to confirm the trade and then confirm the trade of cards, which then the PokéTrader application will swap the corresponding cards in each person's binder to trade.

User View

User Main Page (and Login)

- Users can create a new account.
- Users must log in using their username and password in order to access their account.
- Users can log out.
- After login, the system creates a secure session for the user.
 - The first user using the account who is successfully logged into the system will be the only user who has access to the account. Other users who attempt logging into the same account will be denied access.

Pack-Opening Page

- Users will receive one free ticket per day (i.e. one for every 12 hours). In order to redeem the free packs, users must navigate to the "Pack Opening Page" and use their tickets to redeem a certain pack.
- Users can change their Chase Pack (which pack they want to open).
- On the top left of the page, there will be a message letting the user know when the next ticket can be earned via a timer.

Binder Page

- Users can use the PokéTrader sorting system to display a subset of cards in the user's binder collection. This binder sorting system which will include the following:
 - A search bar
 - The user can type in the search bar to search (**case-insensitive**) these particular characteristics:
 - Name
 - e.g. "charizard" → looks for cards named "Charizard"
 - "name = charizard" also works
 - Card-Type

- e.g. “card-type = pokemon” → looks for only Pokémon cards
 - e.g. “card-type = item” → looks for only Item cards
 - e.g. “card-type = energy” → looks for only Energy cards
- Pokémon-Type
 - e.g. “pokemon-type = grass” → looks for only Pokémon cards that are Grass-types
- Expansion
 - e.g. “set = SVI” → looks for cards only in the Scarlet and Violet set
 - “set = svi” also works
- An “Advanced Options” button
 - Pulls up a selection of buttons to filter the following:
 - Card-Type
 - Pokémon
 - Trainer
 - Energy
 - Pokémon-Type
 - Grass, Fire, Water, Lightning, Psychic, Fighting, Darkness, Metal, Dragon, and Colorless
 - Expansion
 - Represented as 2-3 characters
 - e.g. Paldean Fates → PAF
 - e.g. Diamond & Pearl → DP
 - Rarity
 - Common (Black Circle ○)
 - Uncommon (Black Diamond ◇)
 - Rare (Black Star ★)
 - Double Rare (Double Black Star ★★)
 - Ultra Rare (Silver/Clear Stars ★★)
 - Illustration Rare (Single Gold Star ★)
 - Special Illustration Rare (Double Gold Stars ★★)
 - Hyper Rare (Triple Gold Stars ★★★)
 - Promo (Black Star with “PROMO”)
- Users also have the option to flip through their binder collection by clicking either a “right arrow” button on the right side of the binder or a “left arrow” button on the left side of the binder.

Trade Page

- Initiate a new trade request

- The first user will fill out a short form that details what is being offered as a trade (i.e. one or more cards) and can optionally put what cards they want to trade for (status of the trade becomes Request).
- Accept a trade request
 - The second user will either decline the trade, indicating that they will not trade the card (status of the trade becomes Declined), or accept and request one or more cards from the first user (status of the trade becomes Pending Response).
 - The first user will either decline the trade, indicating that they are unwilling to continue with the trade (status of the trade becomes Declined), or accept and select one of the options that the second user requested.
 - The second user can decline the trade, indicating that they are unwilling to continue with the trade (status of the trade becomes Declined). Otherwise, the second user can accept the trade, to which both users are now in agreement, meaning the transaction will occur (status of the trade becomes Accepted).

Explanation of Entity Sets & Relationships

Entity Sets (6)

Users

- PK: Identified by the **username**.
- FK: The **packID** from *Pack* to represent the *Opens* relation.
- Attributes: A login **passwordHash** string (after the user's password is hashed) and a binary value to determine if they are **loggedIn** to the system. The number of **tickets** for redeeming a free pack from any available *Packs* can be earned by waiting until a designated time (**nextTicketTime**).
- Relations:
 - *Performs*: A *User* has a many-to-many relationship with *Trades*.
 - *Owns*: A *User* has a many-to-many relationship with *Cards*.
 - *Involves*: A *User* has a ternary many-many-many relationship with *Cards* and *Trades* (this is distinct from *Performs* and *Owns* because it serves a different purpose)
 - *Writes*: A *User* has a one-to-many relationship with *TradeMessages*.

Sets

- PK: Identified by the **setCode** (defined by three characters).
- Attributes: The **setName** string for human-friendly identification.
- Relations:
 - *PackFromSet*: A *Set* has a one-to-many relationship with *Packs*.
 - *CardFromSet*: A *Set* has a one-to-many relationship with *Cards*.

Packs

- PK: Identified by the **packID**.
- FK: The **setCode** from *Set* to represent the *PackFromSet* relation.
- Attributes: The **packName** string and a boolean representing whether the *Pack* is currently **available** for opening.
- Relations:
 - *Contains*: A *Pack* has a many-to-many relationship with *Cards*.
 - *PackFromSet*: A *Pack* has a one-to-many relationship with *Sets*.

Cards

- PK: Identified by the **cardID**.
- FK: The **setCode** from *Set* to represent the *CardFromSet* relation.
- Attributes: The **cardName**, Pokemon/Trainer **cardType**, card **subType** (e.g. Water, Fire, Item, Stadium), card **rarity**, and an **imageLink** to the card art.
- Relations:
 - *Contains*: A *Card* has a many-to-many relationship with *Packs*.
 - *CardFromSet*: A *Card* has a one-to-many relationship with *Sets*.
 - *Owns*: A *Card* has a many-to-many relationship with *Users*.
 - *Involves*: A *Card* has a ternary many-many-many relationship with *Users* and *Trades*.

Trades

- PK: Identified by the **tradeID**.
- Attributes: The **status** of the trade as Accepted, Denied, Pending response, and Request state.
- Relations:
 - *Has*: A *Trade* supports a weak one-to-many relationship with *TradesMessages*.
 - *Involves*: A *Trade* has a ternary many-many-many relationship with *Users* and *Cards*.
 - *Performs*: A *Trade* has a many-to-many relationship with exactly 2 *Users*.

TradeMessages

- PK: Identified by the **messageNum**.
- FK: Supported by the **tradeID** of the *Trade*.
- Attributes: The **timestamp** of the messages are recorded and the contents of the **message**.
- Relations:
 - *Has*: *TradeMessages* is supported by *Trade* because a message cannot be sent regarding a *Trade* without the original *Trade* existing.
 - *Writes*: A *TradeMessage* has a one-to-many relationship with *Users*.

Relations (4)

Performs

The *Performs* relationship connects the *Users* and *Trades* through a many-to-many relationship. A *User* can *Perform* many *Trades*, and a *Trade* can be *Performed* by exactly 2 *Users* (hence, we use the many designation). It will involve the **username** of the *User* and **tradeID** of the *Trade* as foreign keys to make up the table.

Involves

The *Involves* ternary relationship connects the *Users*, *Trades*, and *Cards* through a many-many-many relationship. A *Trade Involves* many *Cards* and many *Users* (exactly 2 as specified in *Performs*). A *Card* is associated with many *Users* (across different *Trades*) and can be *Involved* in many *Trades*. A *User* is associated with many *Cards* (across different *Trades*) and can be *Involved* in many *Trades*. It will involve the **tradeID** of the *Trade*, the **cardName** of the *Card*, and the **username** of the *User* as foreign keys to make up the table. The **quantity** requested will also be specified in this relationship.

Owns

The *Owns* relationship connects the *Users* and *Cards* through a many-to-many relationship. A *User* can *Own* many *Cards*, and a *Card* can be *Owned* by many *Users*. It will involve **username** of the *User* and **cardName** of the *Card* as foreign keys to make up the table as well as the **quantity** owned and how many copies of a *Card* (**numberOffered**) the *User* wants to offer for trades.

Contains

The *Contains* relationship connects the *Packs* and *Cards* through a many-to-many relationship. A *Pack* can *Contain* many *Cards*, and a *Card* can be *Contained* in many *Packs*. It will involve **packID** of the *Pack* and **cardID** of the *Card* as foreign keys to make up the table.

Relations without Table (4)

PackFromSet

The *PackFromSet* relationship connects the *Packs* that belong to a *Set* through a one-to-many relationship. A *Pack* can only belong to one *Set*, but a *Set* can have multiple unique *Packs*. This will be represented by a foreign key under the *Packs* table referencing the *Set* it belongs to.

CardFromSet

The *CardFromSet* relationship connects the *Cards* that belong to a *Set* through a one-to-many relationship. A *Card* can only belong to one *Set*, but a *Set* can have many unique *Cards*. This will be represented by a foreign key under the *Cards* table referencing the *Set* it belongs to.

Has

The *Has* relationship connects the weak entity set *TradeMessages* to the *Trade* they are associated with through a one-to-many relationship. *TradeMessages* only exist if the original, singular *Trade* it belongs to remains stored in history. A *Trade* can have numerous back and forth *TradeMessages*. This will be represented by a foreign key under the *TradeMessages* table referencing the *Trade* it comes from, specifically, the primary key **tradeID** from *Trade*.

Writes

The *Writes* relationship connects the *TradeMessages* to *Users* through a one-to-many relationship. A *TradeMessage* is written by exactly one *User*, but a *User* can write many *TradeMessages*. This will be represented by a foreign key under the *TradeMessages* table referencing the *User* it was written by.

Relational Schema

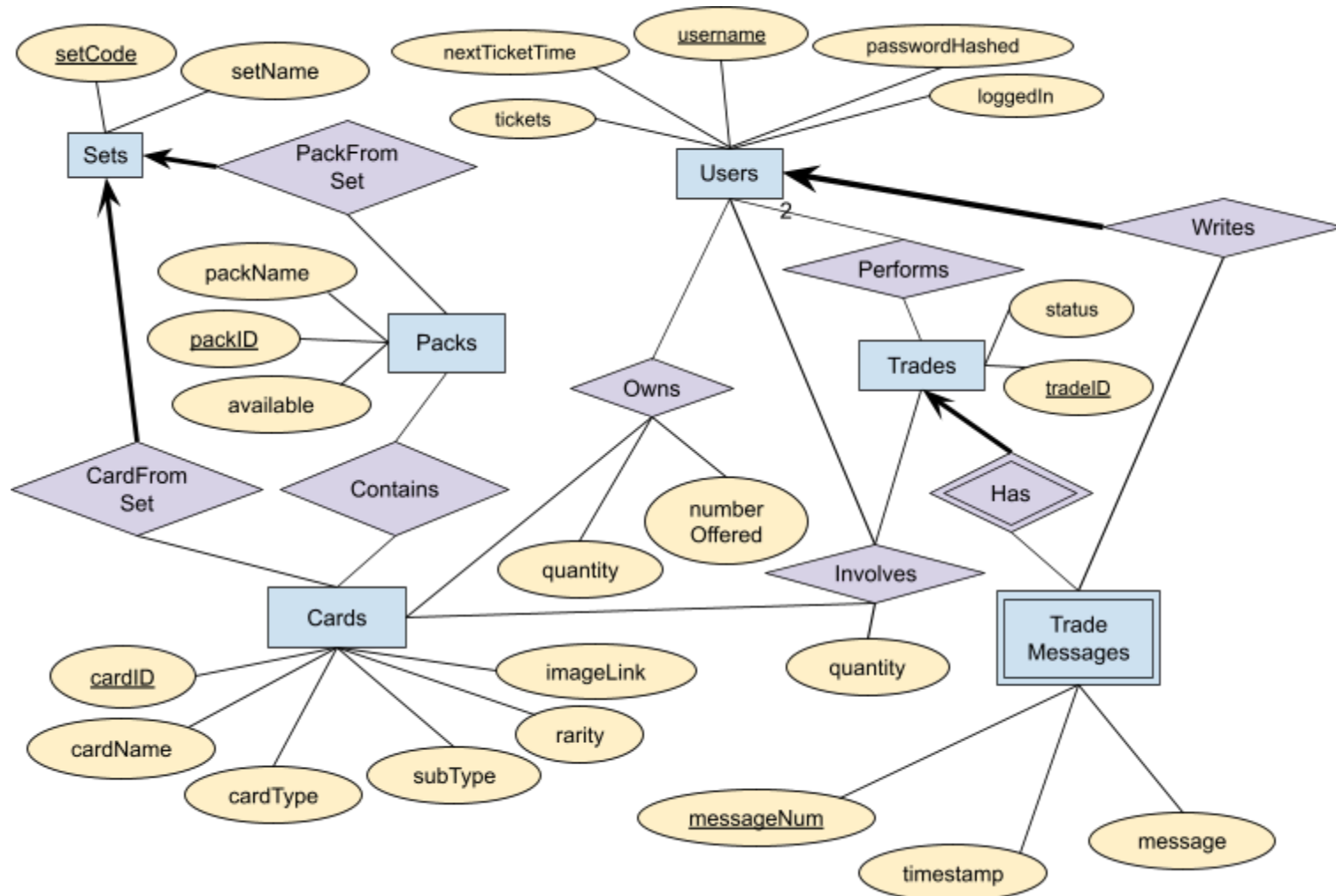
Entity Sets

- Users(username, passwordHashed, loggedIn, nextTicketTime, tickets)
- Sets(setCode, setName)
- Packs(packID, packName, available, *setCode*)
- Cards(cardID, cardName, rarity, cardType, subType, imageLink, *setCode*)
- Trades(tradeID, status)
- TradeMessages(messageNum, tradeID, message, timestamp, *username*)

Relations

- Performs(username, tradeID)
- Involves(cardID, tradeID, username, quantity)
- Owns(username, cardID, quantity, numberOffered)
- Contains(packID, cardID)

E/R Diagram



Workbench Screenshots (refer to old ERD)

Users

MySQL Workbench

Local Server - Warning - not s... x

File Edit View Query Database Server Tools Scripting Help

Navigator

SQL File 4* x users - Table

Limit to 1000 rows

```
1 SELECT *
2 FROM users;
```

Result Grid

	username	password	loggedIn	nextTicketTime	tickets	packID
▶	alex01	V7!qR2#nL9@x	1	2026-01-15 08:30:00	3	1
	casey_admin	R9&dM5#yX2!k	0	2026-04-09 06:25:48	3	4
	drew456	C1!sY7#fG4@q	1	2026-09-19 13:35:42	3	9
	jamie_test	VW6@rD3%uN8^p	1	2026-08-12 09:05:55	3	8
	jordan_dev	N6@hQ1%vC8!s	0	2026-03-22 19:10:35	3	3
	morgan22	T4\$zK8!pW3^b	1	2026-02-03 14:45:12	0	2
	quinn_guest	M5\$kT2!zA9&v	1	2026-10-01 17:50:18	3	10
	riley_user	H8^nP4!cB1@t	1	2026-05-17 11:55:20	3	6
	sam_database	K2#xV9\$eJ5!m	1	2026-06-28 16:40:05	3	7
	taylor78	F3!aL7@qZ6\$w	1	2026-04-09 06:25:48	3	5
*	NULL	NULL	NULL	NULL	NULL	NULL

users 29 x

Output

Action Output

#	Time	Action
✓ 43	12:01:00	UPDATE cards SET setCode = 'HON' WHERE cardID = '02012'

Sets

MySQL Workbench

Local Server - Warning - not s... x

File Edit View Query Database Server Tools Scripting He

SQL File 4* x users - Table

Navigator:

SCHEMAS

Filter objects

- hw1
- ko
- poketrader
- poketrader_blueprint**
 - Tables
 - cards
 - contains
 - involves
 - owns
 - packs
 - performs
 - sets**
 - Columns
 - setCode
 - setName
 - Indexes
 - Foreign Keys
 - Triggers
 - trademessages
 - trades
 - users
 - Views
 - Stored Procedures
 - Functions
- sys

SQL File 4* x users - Table

```
1 SELECT *
2 FROM sets;
```

Result Grid | Filter Rows:

	setCode	setName
▶	ALO	Aloha
	GLR	Galar
	HON	Hoenn
	JOH	Johto
	KAL	Kalos
	KAN	Kanto
	PAL	Paldea
	SIN	Sinnoh
	UNO	Unova
	WNW	Singapore
*	NULL	NULL

sets 30 x

Packs

MySQL Workbench

The screenshot shows the MySQL Workbench interface. The top menu bar includes File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. Below the menu is a toolbar with various icons. The main window is titled 'Local Server - Warning - not s...' and contains a 'Navigator' pane on the left and a 'SQL File 4*' editor on the right.

The 'Navigator' pane shows a tree of schemas. The 'pocketrader_blueprint' schema is expanded, showing a table named 'packs'. The 'packs' table is selected, and its columns are listed: packID, packName, available, and setCode.

The 'SQL File 4*' editor contains the following SQL query:

```
1 SELECT *
2 FROM packs;
```

The 'Result Grid' pane shows the results of the query. It contains a table with 10 rows and 4 columns: packID, packName, available, and setCode. The first 10 rows are highlighted in blue. The last row is a summary row with NULL values.

	packID	packName	available	setCode
▶	1	Red	0	KAN
	2	Green	0	KAN
	3	Blue	0	KAN
	4	Soulsilver	1	JOH
	5	Heartgold	1	JOH
	6	Primal Ruby	1	HON
	7	Primal Sapphire	1	HON
	8	Primal Emerald	1	HON
	9	Diamond	1	SIN
	10	Pearl	1	SIN
*	NULL	NULL	NULL	NULL

Cards

MySQL Workbench

Local Server - Warning - not s...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- hw1
- ko
- poketrader
 - poketrader_blueprint
 - Tables
 - cards
 - Columns
 - cardID
 - cardName
 - rarity
 - cardType
 - subType
 - imageLink
 - setCode
 - Indexes
 - Foreign Keys
 - Triggers
 - contains
 - involves
 - owns
 - packs
 - Columns
 - packID
 - packName
 - available
 - PackFromSet
 - Indexes
 - Foreign Keys
 - Triggers
 - performs

Administration Schemas

SQL File 4* cards - Table

Limit to 1000 rows

Result Grid

cardID	cardName	rarity	cardType	subType	imageLink	setCode
00001	Bulbasaur	Common	Pokemon	Grass	https://assets.pokemon.com/static-assets/cont...	KAN
00002	Blastoise	Rare	Pokemon	Water	https://assets.pokemon.com/static-assets/cont...	KAN
00023	Misty	Uncommon	Trainer	Supporter	https://assets.pokemon.com/static-assets/cont...	KAN
01001	Cyndaquil	Common	Pokemon	Fire	https://assets.pokemon.com/static-assets/cont...	JOH
01004	Chikorita	Common	Pokemon	Grass	https://assets.pokemon.com/static-assets/cont...	JOH
01085	Magnetic Storm	Uncommon	Trainer	Stadium	https://assets.pokemon.com/static-assets/cont...	JOH
02012	Sceptile	Rare Holo	Pokemon	Grass	https://assets.pokemon.com/static-assets/cont...	HON
02090	Oran Berry	Uncommon	Trainer	Pokemon Tool	https://assets.pokemon.com/static-assets/cont...	HON
03101	Raichu LV.X	Rare Holo	Pokemon	Electric	https://assets.pokemon.com/static-assets/cont...	SIN
03102	Regigigas LV.X	Rare Holo	Pokemon	Colorless	https://assets.pokemon.com/static-assets/cont...	SIN
NULL	NULL	NULL	NULL	NULL	NULL	NULL

cards 27 x

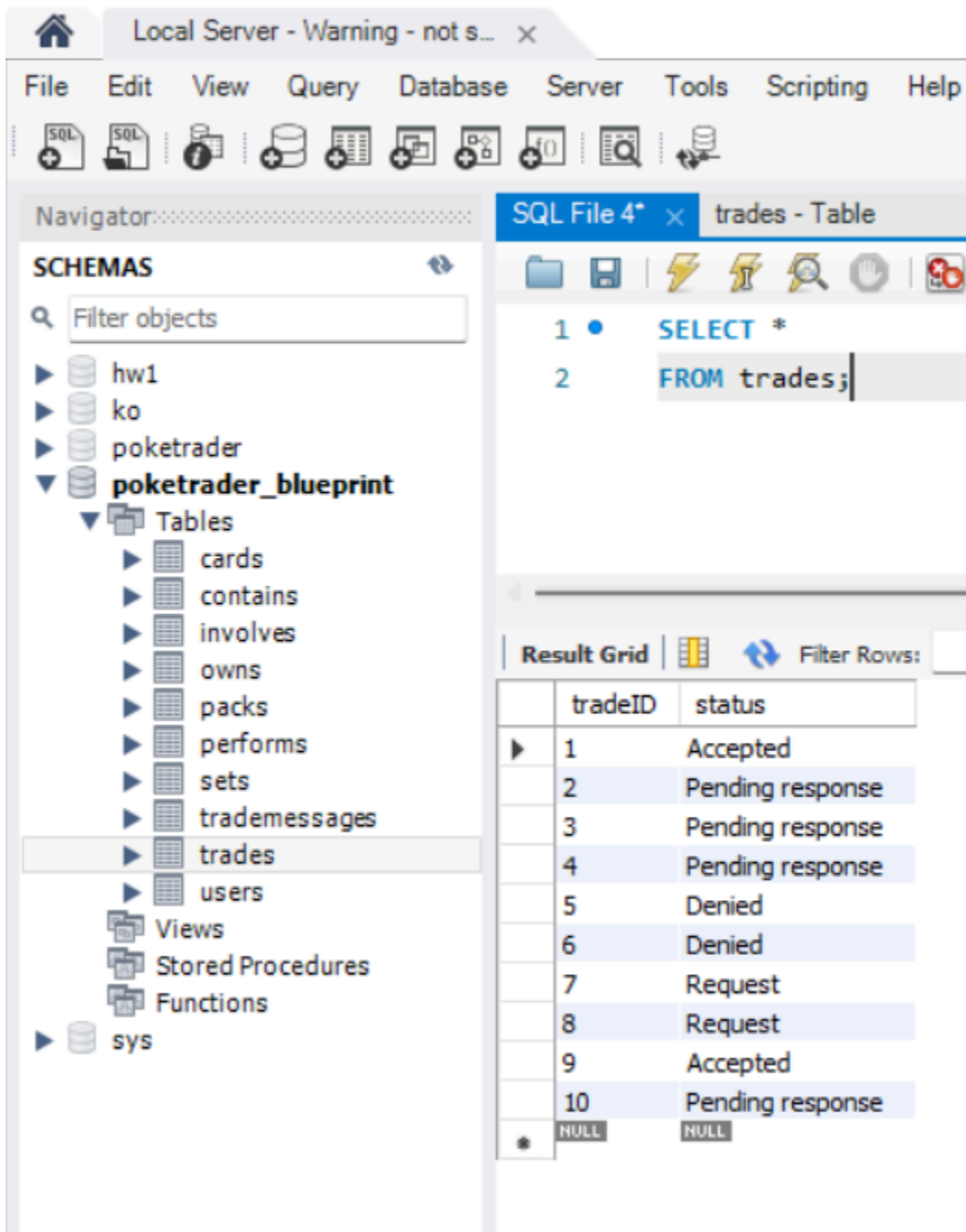
Output

Action Output

#	Time	Action	Message
40	12:00:33	SELECT * FROM cards LIMIT 0, 1000	10 row(s)

Trades

 MySQL Workbench



The screenshot shows the MySQL Workbench interface. The top menu bar includes File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. Below the menu is a toolbar with various icons. The left sidebar, titled 'Navigator', shows a tree of schemas. The 'poketrader_blueprint' schema is expanded, showing tables like cards, contains, involves, owns, packs, performs, sets, trademessages, trades, and users. The 'trades' table is selected. The main window displays a SQL query in a text editor:

```
1 SELECT *
2 FROM trades;
```

Below the query editor is a 'Result Grid' tab. It shows a table with two columns: 'tradeID' and 'status'. The results are as follows:

tradeID	status
1	Accepted
2	Pending response
3	Pending response
4	Pending response
5	Denied
6	Denied
7	Request
8	Request
9	Accepted
10	Pending response
NULL	NULL

TradeMessages

MySQL Workbench

Local Server - Warning - not s... x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- hw1
- ko
- poketrader
- poketrader_blueprint**
 - Tables
 - cards
 - contains
 - involves
 - owns
 - packs
 - performs
 - sets
 - trademessages**
 - Columns
 - messageID
 - tradeID
 - username
 - message
 - timestamp
 - Indexes
 - Foreign Keys
 - Triggers
 - trades
 - users

SQL File 4* x trademessages - Table

Limit to 1000 rows

```
SELECT *  
FROM trademessages;
```

Result Grid

Filter Rows:

Edit: Export/Import: Wrap Cell Conte

	messageID	tradeID	username	message	timestamp
▶	10001	1	alex01	Thanks for the trade!	2026-01-15 08:30:00
	10002	1	morgan22	Yeah, good trade!	2026-01-15 08:35:00
	20001	2	jordan_dev	What would you be interested in for that Oran ...	2026-04-30 08:00:00
	20002	2	casey_admin	Could trade it for your Misty Trainer Supporter, ...	2026-04-30 08:05:00
	30001	3	taylor78	Wow, that's a really nice Raichu ex you got the...	2026-05-01 08:00:00
	30002	3	riley_user	I got pretty lucky with my pulls for the Sinnoh-D...	2026-05-01 08:05:00
	50001	5	sam_database	Whoops, looks like I already have this card in m...	2026-05-01 08:00:00
	50002	5	jamie_test	np, it's all good	2026-05-01 08:05:00
	90001	9	drew456	Can I trade you this Sceptile for your Regigigas ...	2026-05-01 10:00:00
	90002	9	quinn_guest	Sure, sounds like a deal (thumbs up)	2026-05-01 10:05:00
*	NULL	NULL	NULL	NULL	NULL

Performs

MySQL Workbench

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'poketrader_blueprint' database expanded, showing a list of tables including 'performs'. The main window is titled 'SQL File 4*' and 'performs - Table'. Below the toolbar, there is a 'Result Grid' tab. The 'Result Grid' displays the following data:

username	tradeID
alex01	1
morgan22	1
casey_admin	2
jordan_dev	2
riley_user	3
taylor78	3
jamie_test	4
sam_database	4
drew456	5
quinn_guest	5
NULL	NULL

Involves

MySQL Workbench

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with the 'poketrader_blueprint' database expanded, showing tables like 'cards', 'contains', 'involves', 'owns', 'packs', 'performs', 'sets', 'trademessages', 'trades', and 'users'. The 'involves' table is selected. The main editor shows a SQL query: `SELECT * FROM involves;`. The 'Result Grid' at the bottom displays the query results, showing columns 'cardID' and 'tradeID' with various values. The status bar at the bottom indicates 'involves 47 x'.

Local Server - Warning - not s... x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 4* x involves - Table

SCHEMAS

Filter objects

- hw1
- ko
- poketrader
- poketrader_blueprint**
 - Tables
 - cards
 - contains
 - involves**
 - owns
 - packs
 - performs
 - sets
 - trademessages
 - trades
 - users
 - Views
 - Stored Procedures
 - Functions
- sys

10 ('03102', 9),
11 ('01085', 10),
12 ('01004', 10);
13
14 • SELECT *
15 FROM involves;
16

Result Grid | Filter Rows:

	cardID	tradeID
▶	00001	1
	01001	1
	00023	2
	02090	2
	00002	3
	03101	3
	02012	9
	03102	9
	01004	10
	01085	10
•	NULL	NULL

involves 47 x

Owens

MySQL Workbench

The screenshot shows the MySQL Workbench interface. On the left, the 'Navigator' pane displays the 'owns' table under the 'poketrader_blueprint' schema. The table has four columns: 'username', 'cardID', 'quantity', and 'offeredForTrade'. The 'owns' table is expanded, showing its columns and indexes.

In the center, the 'SQL File 4*' pane shows a query being executed:

```
11      ('drew456', '03102', 1, False),
12      ('quinn_quest', '02012', 1, False)
13
14 •  SELECT *
15      FROM owns;
16
```

Below the SQL editor, the 'Result Grid' pane displays the query results. The results are shown in a table with four columns: 'username', 'cardID', 'quantity', and 'offeredForTrade'. The results include data for various users and their card holdings.

username	cardID	quantity	offeredForTrade
alex01	1001	1	0
casey_admin	2090	1	1
drew456	3102	1	0
jamie_test	1004	1	0
jordan_dev	23	1	1
morgan22	1	1	0
quinn_quest	2012	1	0
riley_user	3101	1	1
sam_database	1085	1	0
taylor78	2	1	1
*	NULL	NULL	NULL

Contains

MySQL Workbench

The screenshot shows the MySQL Workbench interface. The left sidebar (Navigator) displays the database schema, with the 'contains' table selected under the 'poketrader_blueprint' database. The main window shows a query in the SQL editor:

```
9      (3, '02012'),
10     (3, '02090'),
11     (4, '03101'),
12     (4, '03102');
13
14 •  SELECT *
15     FROM contains
```

Below the SQL editor, the 'Result Grid' tab is active, displaying the query results in a table:

	packID	cardID
▶	1	00001
	1	00002
	1	00023
	2	01001
	2	01004
	2	01085
	3	02012
	3	02090
	4	03101
	4	03102
•	NULL	NULL